МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

Отчет по лабораторной работе №2
по курсу
«Нейронные сети и машинное обучение»

Выполнила:

студент группы 6231-090401D

Ризванова Э. Т.

Проверила:

к.т.н., доцент Солдатова О.П.

Самара 2025

СОДЕРЖАНИЕ

Введение 3

1 Порядок выполнения работы 4

2 Считывание данных для классификации изображений 6

3 Создание модели свёрточной нейронной сети 7

4 Исследование зависимости функции ошибки и accuracy от числа эпох обучения 8

5 Исследование зависимости функции ошибки и accuracy от размера мини-батча 12

6 Исследование зависимости функции ошибки и accuracy от уровня исключения нейронов 16

7 Результаты исследований 20

Заключение 22

Список использованных источников 23

Приложение А. Листинг программы 24

ВВЕДЕНИЕ

Целью лабораторной работы «Решение задачи классификации изображений на примере решения задачи распознавания дефектов поверхности металлов на изображениях в оттенках серого цвета» является демонстрация способности нейронной сети решать задачи классификации на примере решения задачи распознавания дефектов поверхности металлов на изображениях в оттенках серого цвета (определение одного из 6 типов дефектов).

Для реализации задачи используется модель свёрточной нейронной сети с использованием библиотек Keras и Tensorflow и языка Python.

Набор данных для классификации – NEU Metal Surface Defects Data.

Вариант задания для выполнения лабораторной работы приведен в таблице 1.

Таблица 1 – Вариант задания для выполнения лабораторной работы 1

№	Алгоритм 1	Алгоритм 2	Число слоёв свёртки	Число слоёв субдискретизации	Число слоёв dropout	Функция активации	Функция ошибки
10	Adamax	Nadam	3	2	1	PReLU	sparse categorical crossentropy

1 Порядок выполнения работы

1) считать файл с данными для классификации изображений на примере решения задачи распознавания дефектов поверхности металлов на изображениях в оттенках серого цвета (определение одного из 6 типов дефектов);

2) написать программу, имитирующую работу свёрточной сети с настраиваемым числом слоёв в соответствии с вариантом задания;

3) провести обучение сети с использованием заданных алгоритмов обучения SGD с импульсом (моментом) Нестерова, RMSProp, Adam, Adamax, Nadam, Adafactor, AdamW;

4) провести тестирование сети, обученной при помощи всех алгоритмов;

5) продемонстрировать работу свёрточной сети преподавателю;

6) исследовать зависимость функции ошибки при обучении и тестировании, а также метрики оценки качества accuracy от числа эпох обучения для заданных алгоритмов;

7) исследовать зависимость функции ошибки при обучении и тестировании, а также метрики оценки качества accuracy от размера мини-батча;

8) исследовать зависимость функции ошибки при обучении и тестировании, а также метрики оценки качества accuracy от уровня исключения нейронов (dropout изменять в диапазоне 0,1-0,5) для заданных алгоритмов;

9) построить графики функции ошибки при обучении и тестировании, а также метрики качества accuracy для всех исследований;

10) рассчитать метрики precision, recall, TPN, FPN, F1;

11) рассчитать и построить графическое изображение для Confusion matrics;

12) сделать вывод о лучшем алгоритме обучения и лучших параметрах топологии сети и обучения;

13) составить отчёт, который должен содержать постановку задачи, текст программы, результаты исследований по пунктам 6-12 и сдать его преподавателю.

2 Считывание данных для классификации изображений

В работе используется набор данных NEU Metal Surface Defects Data, разделённый на обучающую, валидационную и тестовую выборки [1]. Размер изображений – 200x200.

Считывание данных с помощью ImageDataGenerator показано на рисунке 1.

```
def load_dataset(path, batch_size, target_size=(200,200)):
    datagen = ImageDataGenerator(rescale=1./255)

    train_data = datagen.flow_from_directory(
        directory=os.path.join(path, 'train'),
        target_size=target_size,
        color_mode='grayscale',
        class_mode='sparse',
        batch_size=batch_size,
        shuffle=True
    )

    valid_data = datagen.flow_from_directory(
        directory=os.path.join(path, 'valid'),
        target_size=target_size,
        color_mode='grayscale',
        class_mode='sparse',
        batch_size=batch_size,
        shuffle=False
    )

    test_data = datagen.flow_from_directory(
        directory=os.path.join(path, 'test'),
        target_size=target_size,
        color_mode='grayscale',
        class_mode='sparse',
        batch_size=batch_size,
        shuffle=False
    )

    return train_data, valid_data, test_data
```

Рисунок 1 – Считывание набора данных NEU Metal Surface Defects Data

Для нормализации данных используется параметр rescale, который задаётся при создании ImageDataGenerator. Результат считывания представлен на рисунке 2.

```
Found 1656 images belonging to 6 classes.
Found 72 images belonging to 6 classes.
Found 72 images belonging to 6 classes.
```

Рисунок 2 – Результат считывания данных

3 Создание модели свёрточной нейронной сети

Свёрточная нейронная сеть состоит из множества свёрточных слоев, где выходы одних слоёв являются входами для других. Свёртка – линейная операция, поэтому к результатам применяют функции активации, обычно одинаковые для всего слоя [2].

Помимо слоёв активации, в сети могут использоваться:

- полносвязные слои, часто они являются последними слоями;
- слои субдискретизации (пулинг, pooling), которые похожи на свёрточные, но не имеют обучаемых параметров;
- слои dropout, когда в процессе обучения некоторые связи случайно обрываются.

Обучение сети по заданию необходимо провести с помощью алгоритмов Adamax и Nadam.

На рисунке 3 представлена функция создания модели свёрточной нейронной сети согласно варианту.

```
def build_cnn_model(optimizer_name, dropout, learning_rate=0.001, input_shape=(200,200,1)):
    model = models.Sequential()
    model.add(layers.InputLayer(input_shape))

    model.add(layers.Conv2D(32, (3,3)))
    model.add(layers.PReLU())
    model.add(layers.MaxPooling2D((2,2)))

    model.add(layers.Conv2D(64, (3,3)))
    model.add(layers.PReLU())
    model.add(layers.MaxPooling2D((2,2)))

    model.add(layers.Conv2D(128, (3,3)))
    model.add(layers.PReLU())

    model.add(layers.Flatten())
    model.add(layers.Dense(128))
    model.add(layers.PReLU())

    model.add(layers.Dropout(dropout))
    model.add(layers.Dense(6, activation='softmax'))

    if optimizer_name.lower() == 'adamax':
      optimizer = optimizers.Adamax(learning_rate=learning_rate)
    elif optimizer_name.lower() == 'nadam':
      optimizer = optimizers.Nadam(learning_rate=learning_rate)

    model.compile(optimizer=optimizer, loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    print(f"\nМодель создана: optimizer={optimizer_name}, learning_rate={learning_rate}")

    return model
```

Рисунок 3 – Функция создания модели свёрточной нейронной сети

4 Исследование зависимости функции ошибки и accuracy от числа эпох обучения

В данном разделе исследуется влияние числа эпох обучения. Число эпох для исследования: 15, 20, 25, 30. Остальные параметры зададим следующим образом: размер мини-батча – 32, уровень исключения нейронов – 0,3.

Проведём обучение и тестирование модели с алгоритмом обучения Adamax. Результаты исследования приведены в таблице 2.

Таблица 2 – Результаты исследования влияние числа эпох (Adamax)

Метрика	Число эпох			
	15	20	25	30
Training Accuracy	0,9620	0,9656	0,9746	0,9891
Validation Accuracy	0,9444	0,9861	0,9583	0,9444
Test Accuracy	0,9444	0,9583	0,9583	0,9306
Training Loss	0,1198	0,1011	0,0696	0,0491
Validation Loss	0,1214	0,0527	0,0827	0,0491
Test Loss	0,1803	0,1299	0,1463	0,1335

Оптимальное число эпох для Adamax – 20, при котором достигается высокая точность на тестовой выборке и минимальное значение функции ошибки.

Графики аккуратности и функции ошибки для 20 эпох представлены на рисунке 4.

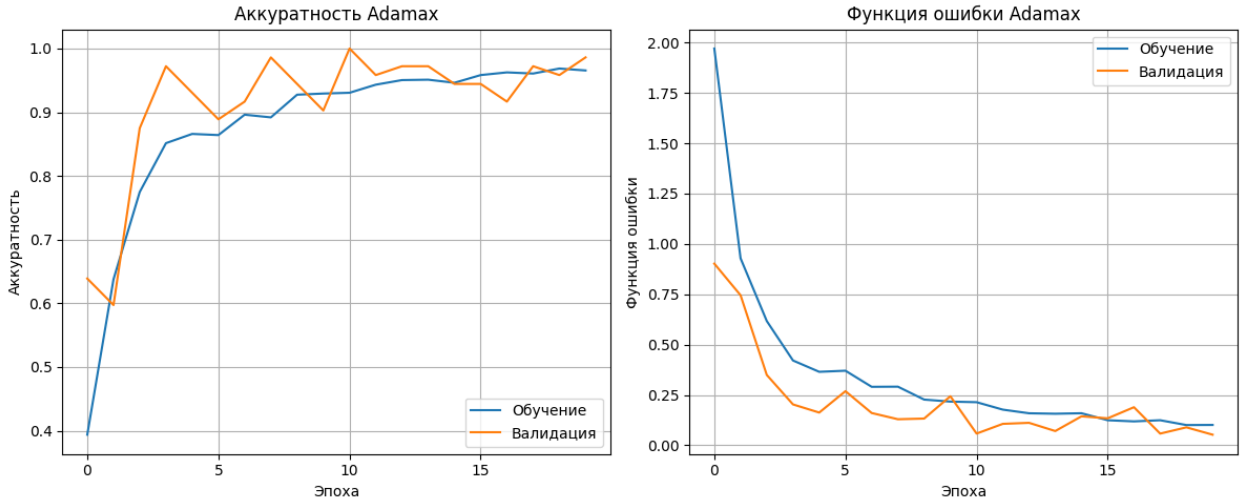

Рисунок 4 – Графики аккуратности и функции ошибки для 20 эпох (Adamax)

Матрица ошибок и отчет о классификации для 20 эпох представлены на рисунках 5 и 6.

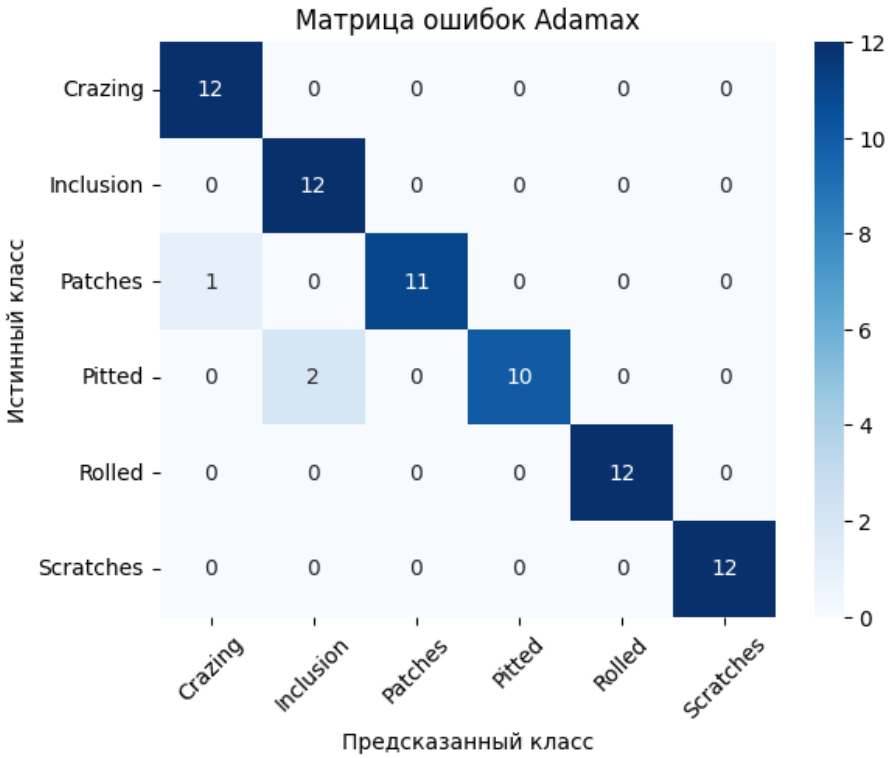

Рисунок 5 – Матрица ошибок для 20 эпох (Adamax)

	precision	recall	f1-score	support
Crazing	0.92	1.00	0.96	12
Inclusion	0.86	1.00	0.92	12
Patches	1.00	0.92	0.96	12
Pitted	1.00	0.83	0.91	12
Rolled	1.00	1.00	1.00	12
Scratches	1.00	1.00	1.00	12
accuracy			0.96	72
macro avg	0.96	0.96	0.96	72
weighted avg	0.96	0.96	0.96	72

Рисунок 6 – Отчет о классификации для 20 эпох (Adamax)

Проведём обучение и тестирование модели с алгоритмом обучения Nadam. Результаты исследования приведены в таблице 3.

Таблица 3 – Результаты исследования влияние числа эпох (Nadam)

Метрика	Число эпох			
	15	20	25	30
Training Accuracy	0,9662	0,9783	0,9861	0,9843
Validation Accuracy	0,9583	0,9583	0,9583	0,9722
Test Accuracy	0,9583	0,9861	0,9444	0,9444
Training Loss	0,1014	0,0723	0,0499	0,0609
Validation Loss	0,0818	0,0836	0,1940	0,0429
Test Loss	0,1566	0,0648	0,1101	0,1900

Оптимальное число эпох для Nadam – 20, при котором достигается высокая точность на тестовой выборке и минимальное значение функции ошибки.

Графики аккуратности и функции ошибки для 20 эпох представлены на рисунке 7.

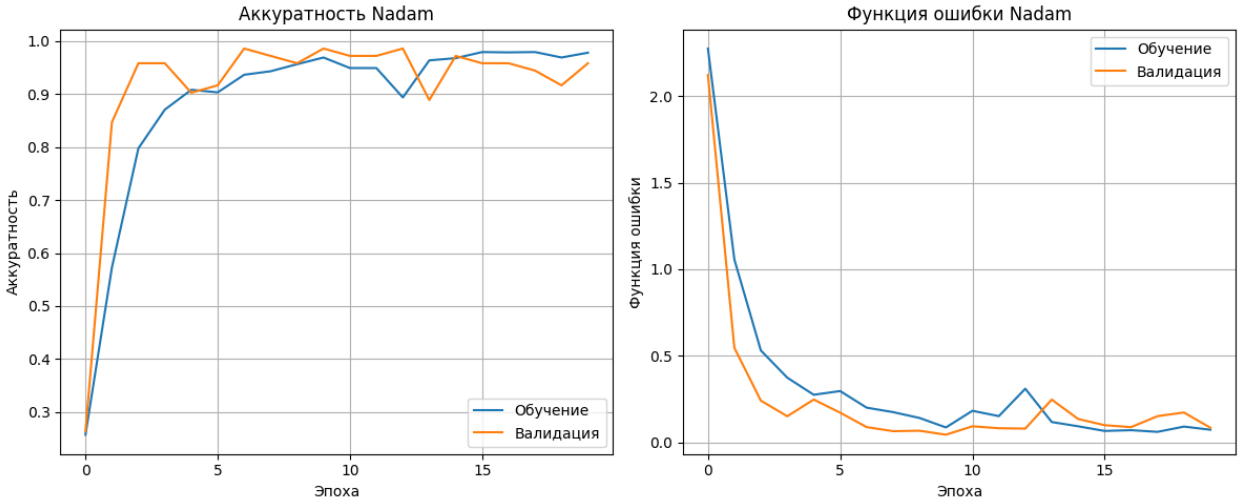

Рисунок 7 – Графики аккуратности и функции ошибки для 20 эпох (Nadam)

Матрица ошибок и отчет о классификации для 20 эпох представлены на рисунках 8 и 9.

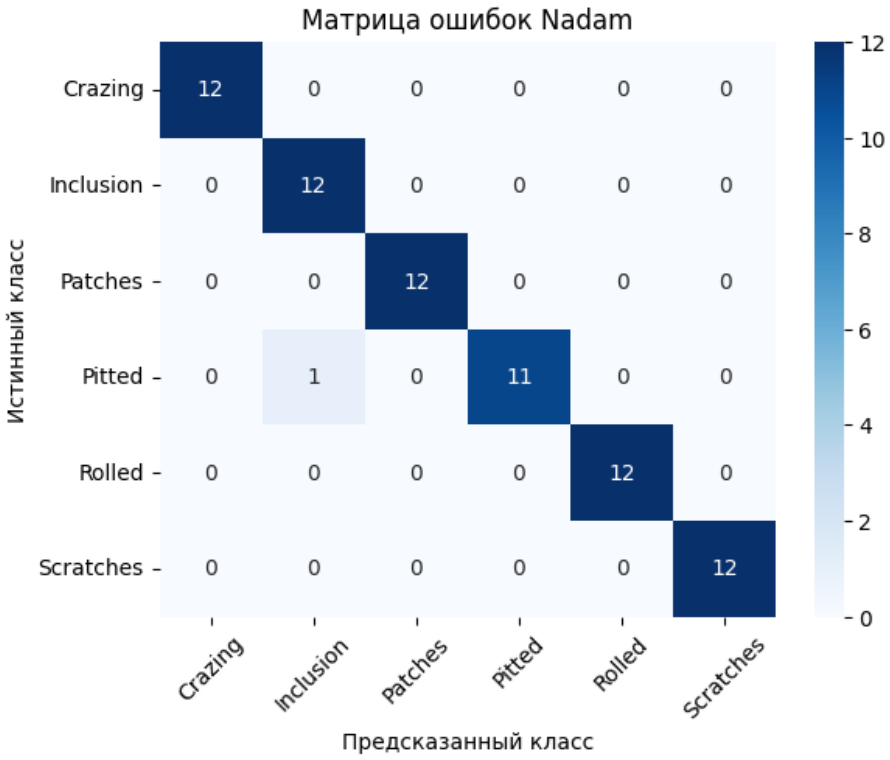

Рисунок 8 – Матрица ошибок для 20 эпох (Nadam)

	precision	recall	f1-score	support
Crazing	1.00	1.00	1.00	12
Inclusion	0.92	1.00	0.96	12
Patches	1.00	1.00	1.00	12
Pitted	1.00	0.92	0.96	12
Rolled	1.00	1.00	1.00	12
Scratches	1.00	1.00	1.00	12
accuracy			0.99	72
macro avg	0.99	0.99	0.99	72
weighted avg	0.99	0.99	0.99	72

Рисунок 9 – Отчет о классификации для 20 эпох (Nadam)

Таким образом, для обоих алгоритмов оптимальным числом эпох является 20.

5 Исследование зависимости функции ошибки и accuracy от размера мини-батча

В данном разделе исследуется влияние размера мини-батча. Размер мини-батча для исследования: 16, 32, 64. Остальные параметры зададим следующим образом: число эпох обучения – 20, уровень исключения нейронов – 0,3.

Проведём обучение и тестирование модели с алгоритмом обучения Adamax. Результаты исследования приведены в таблице 4.

Таблица 4 – Результаты исследования влияние размера мини-батча (Adamax)

Метрика	Размер мини-батча		
	16	32	64
Training Accuracy	0,9686	0,9614	0,9457
Validation Accuracy	0,9583	0,9722	0,9028
Test Accuracy	0,9444	0,9583	0,8889
Training Loss	0,0963	0,1100	0,1608
Validation Loss	0,0835	0,1205	0,2599
Test Loss	0,2047	0,1613	0,3352

Оптимальный размер мини-батча для Adamax – 32, при нём достигается высокая точность на тестовой выборке и минимальное значение функции ошибки.

Графики аккуратности и функции ошибки с размером мини-батча, равным 32, представлены на рисунке 10.

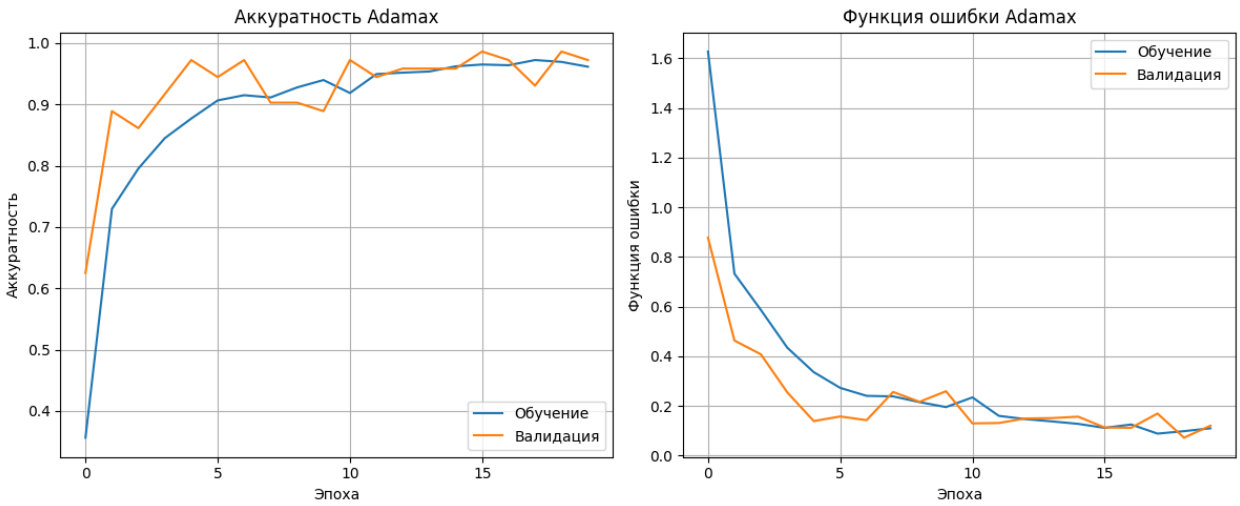

Рисунок 10 – Графики аккуратности и функции ошибки с размером мини-батча, равным 32 (Adamax)

Матрица ошибок и отчет о классификации с размером мини-батча, равным 32, представлены на рисунках 11 и 12.

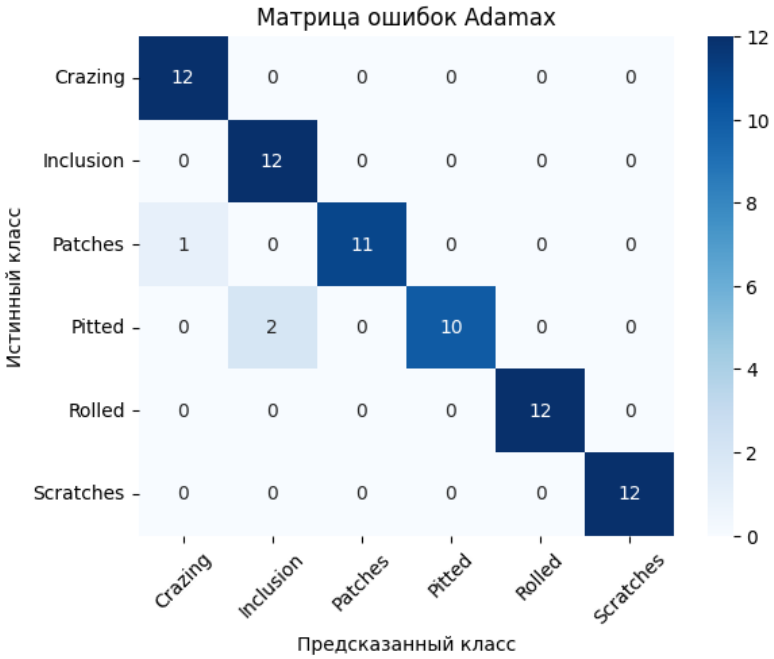

Рисунок 11 – Матрица ошибок с размером мини-батча, равным 32 (Adamax)

	precision	recall	f1-score	support
Crazing	0.92	1.00	0.96	12
Inclusion	0.86	1.00	0.92	12
Patches	1.00	0.92	0.96	12
Pitted	1.00	0.83	0.91	12
Rolled	1.00	1.00	1.00	12
Scratches	1.00	1.00	1.00	12
accuracy			0.96	72
macro avg	0.96	0.96	0.96	72
weighted avg	0.96	0.96	0.96	72

Рисунок 12 – Отчет о классификации с размером мини-батча, равным 32 (Adamax)

Проведём обучение и тестирование модели с алгоритмом обучения Nadam. Результаты исследования приведены в таблице 5.

Таблица 5 – Результаты исследования влияние размера мини-батча (Nadam)

Метрика	Размер мини-батча		
	16	32	64
Training Accuracy	0,9903	0,9897	0,9728
Validation Accuracy	0,9583	0,9444	0,9583
Test Accuracy	0,9583	0,9444	0,9444
Training Loss	0,0330	0,0459	0,0855
Validation Loss	0,0983	0,2954	0,0729
Test Loss	0,0983	0,1057	0,1475

Оптимальный размер мини-батча для Nadam – 16, при нём достигается высокая точность на тестовой выборке и минимальное значение функции ошибки.

Графики аккуратности и функции ошибки с размером мини-батча, равным 16, представлены на рисунке 13.

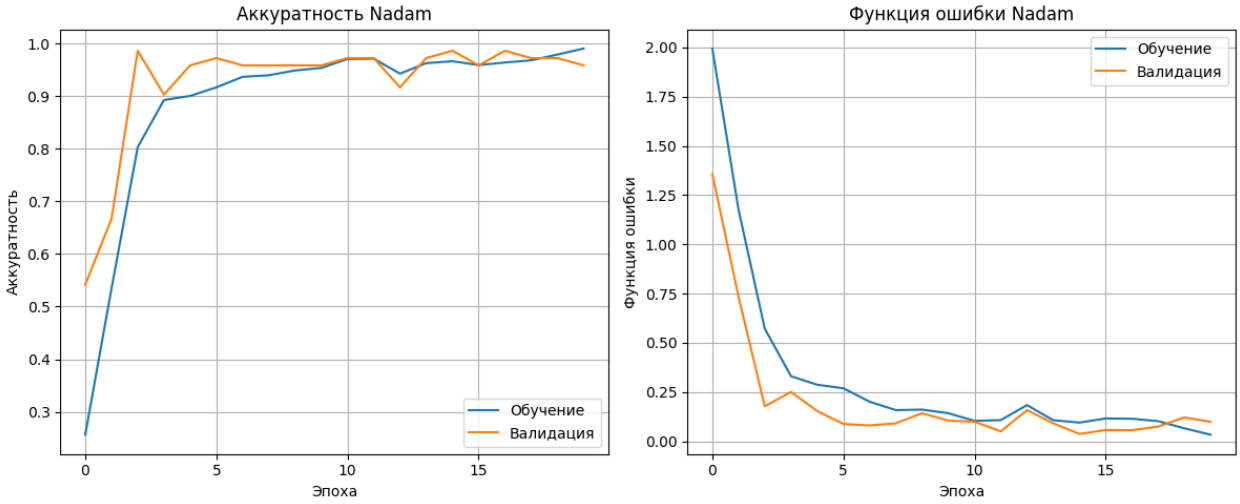

Рисунок 13 – Графики аккуратности и функции с размером мини-батча, равным 16 (Nadam)

Матрица ошибок и отчет о классификации с размером мини-батча, равным 16, представлены на рисунках 14 и 15.

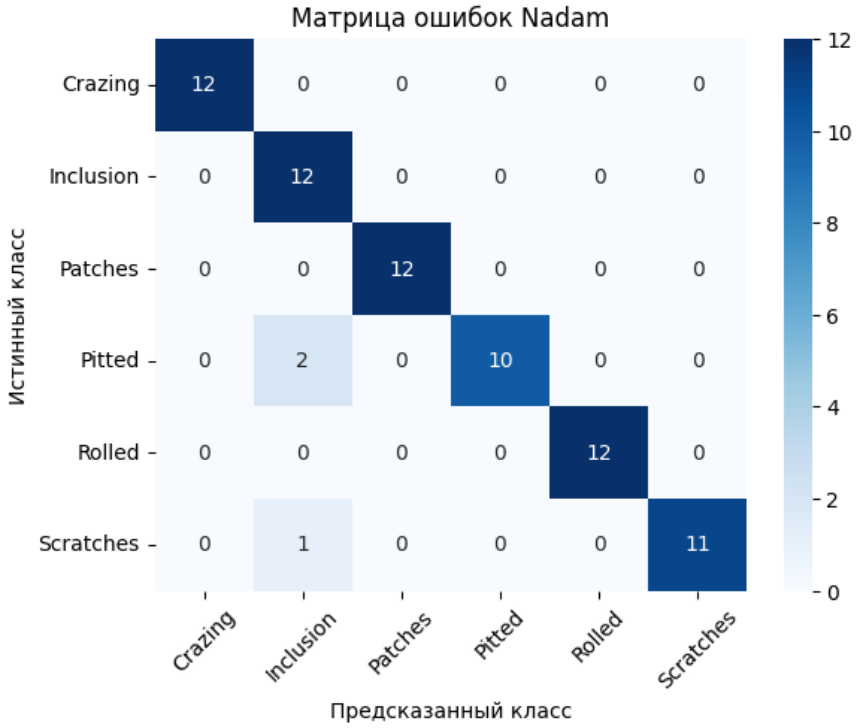

Рисунок 14 – Матрица ошибок с размером мини-батча, равным 16 (Nadam)

	precision	recall	f1-score	support
Crazing	1.00	1.00	1.00	12
Inclusion	0.80	1.00	0.89	12
Patches	1.00	1.00	1.00	12
Pitted	1.00	0.83	0.91	12
Rolled	1.00	1.00	1.00	12
Scratches	1.00	0.92	0.96	12
accuracy			0.96	72
macro avg	0.97	0.96	0.96	72
weighted avg	0.97	0.96	0.96	72

Рисунок 15 – Отчет о классификации с размером мини-батча, равным 16 (Nadam)

Таким образом, для Adamax оптимальным размером мини-батча является 32, а для Nadam – 16.

6 Исследование зависимости функции ошибки и accuracy от уровня исключения нейронов

В данном разделе исследуется влияние уровня исключения нейронов (dropout). Уровень исключения нейронов для исследования: 0,1; 0,2; 0,3; 0,4. Остальные параметры зададим следующим образом: число эпох обучения – 20, размер мини-батча – 32.

Проведём обучение и тестирование модели с алгоритмом обучения Adamax. Результаты исследования приведены в таблице 5.

Таблица 5 – Результаты исследования влияние dropout (Adamax)

Метрика	Уровень исключения нейронов			
	0,1	0,2	0,3	0,4
Training Accuracy	0,9801	0,9553	0,9710	0,9523
Validation Accuracy	0,9583	0,9444	0,9722	0,9444
Test Accuracy	0,9444	0,9306	0,9444	0,9306
Training Loss	0,0758	0,1322	0,0894	0,1389
Validation Loss	0,1770	0,0943	0,0621	0,1102
Test Loss	0,1518	0,2536	0,1446	0,2242

Оптимальный уровень исключения нейронов для Adamax – 0,3, при нём достигается высокая точность на тестовой выборке и минимальное значение функции ошибки.

Графики аккуратности и функции ошибки с dropout=0,3 представлены на рисунке 16.

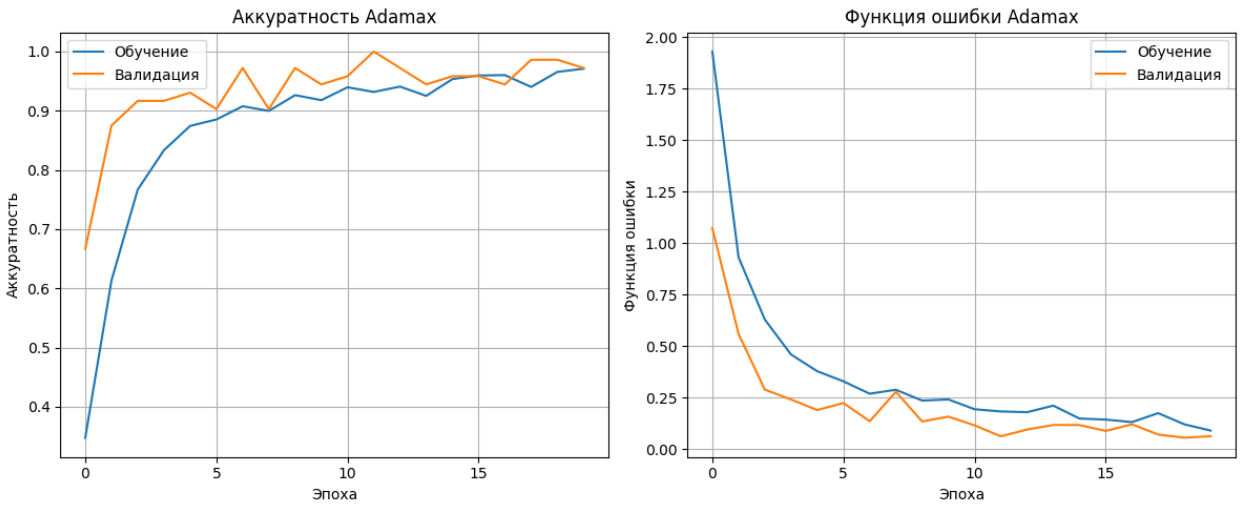

Рисунок 16 – Графики аккуратности и функции с dropout=0,3 (Adamax)

Матрица ошибок и отчет о классификации с dropout=0,3 представлены на рисунках 17 и 18.

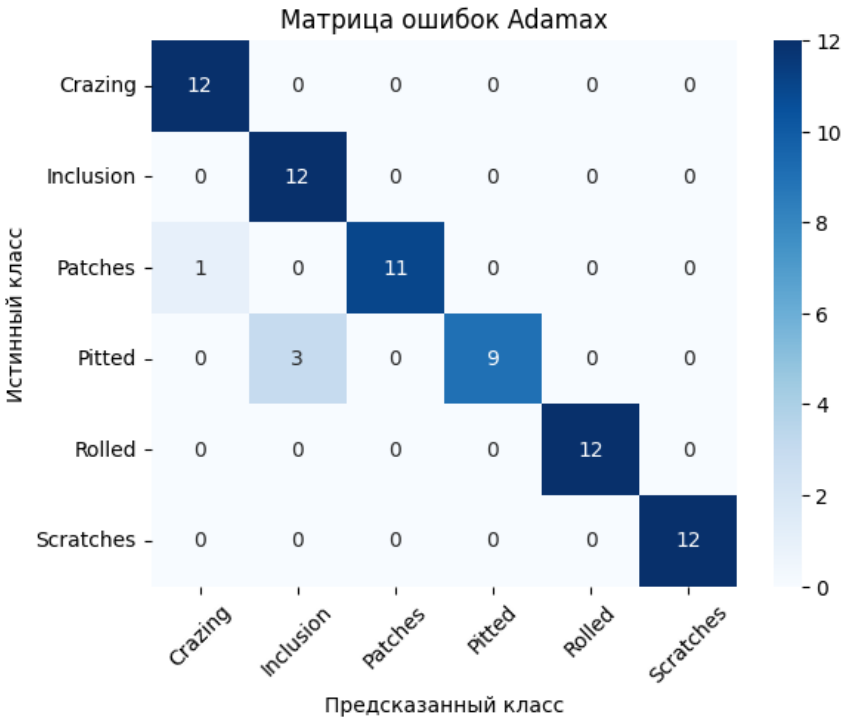

Рисунок 17 – Матрица ошибок с dropout=0,3 (Adamax)

	precision	recall	f1-score	support
Crazing	0.92	1.00	0.96	12
Inclusion	0.80	1.00	0.89	12
Patches	1.00	0.92	0.96	12
Pitted	1.00	0.75	0.86	12
Rolled	1.00	1.00	1.00	12
Scratches	1.00	1.00	1.00	12
accuracy			0.94	72
macro avg	0.95	0.94	0.94	72
weighted avg	0.95	0.94	0.94	72

Рисунок 18 – Отчет о классификации с dropout=0,3 (Adamax)

Проведём обучение и тестирование модели с алгоритмом Nadam (размер мини-батча – 16). Результаты исследования приведены в таблице 6.

Таблица 6 – Результаты исследования влияние dropout (Nadam)

Метрика	Уровень исключения нейронов			
	0,1	0,2	0,3	0,4
Training Accuracy	0,9662	0,9801	0,9728	0,9644
Validation Accuracy	0,9722	0,9722	0,9583	0,9861
Test Accuracy	0,9722	0,9444	0,9444	0,9722
Training Loss	0,1038	0,0601	0,0977	0,1188
Validation Loss	0,0880	0,0452	0,1070	0,0724
Test Loss	0,1362	0,3314	0,2537	0,0927

Оптимальный уровень исключения нейронов для Nadam – 0,4, при нём достигается высокая точность на тестовой выборке и минимальное значение функции ошибки.

Графики аккуратности и функции ошибки с dropout=0,4 представлены на рисунке 19.

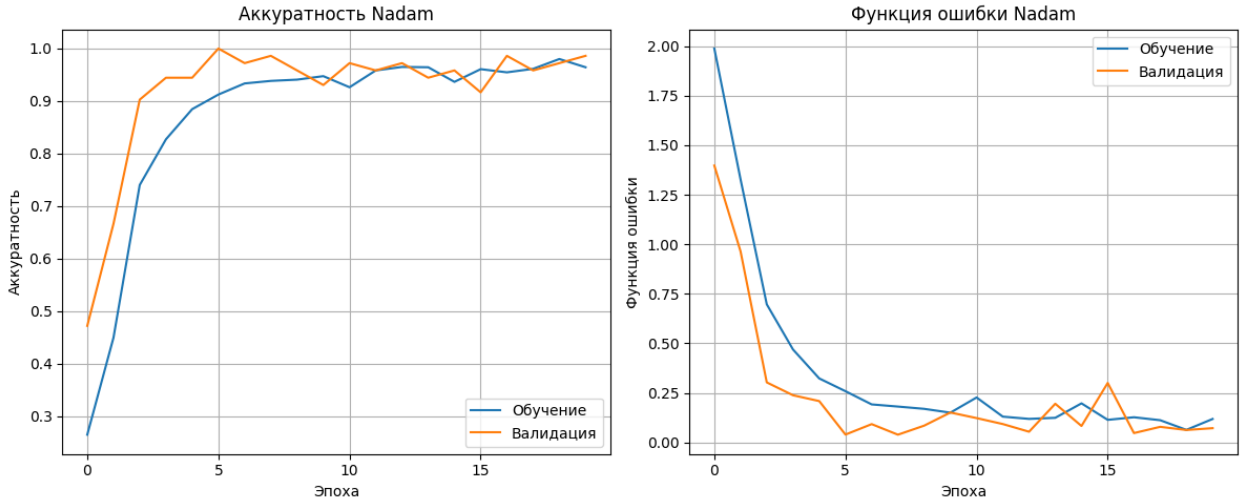

Рисунок 19 – Графики аккуратности и функции с dropout=0,4 (Nadam)

Матрица ошибок и отчет о классификации с dropout=0,4 представлены на рисунках 20 и 21.

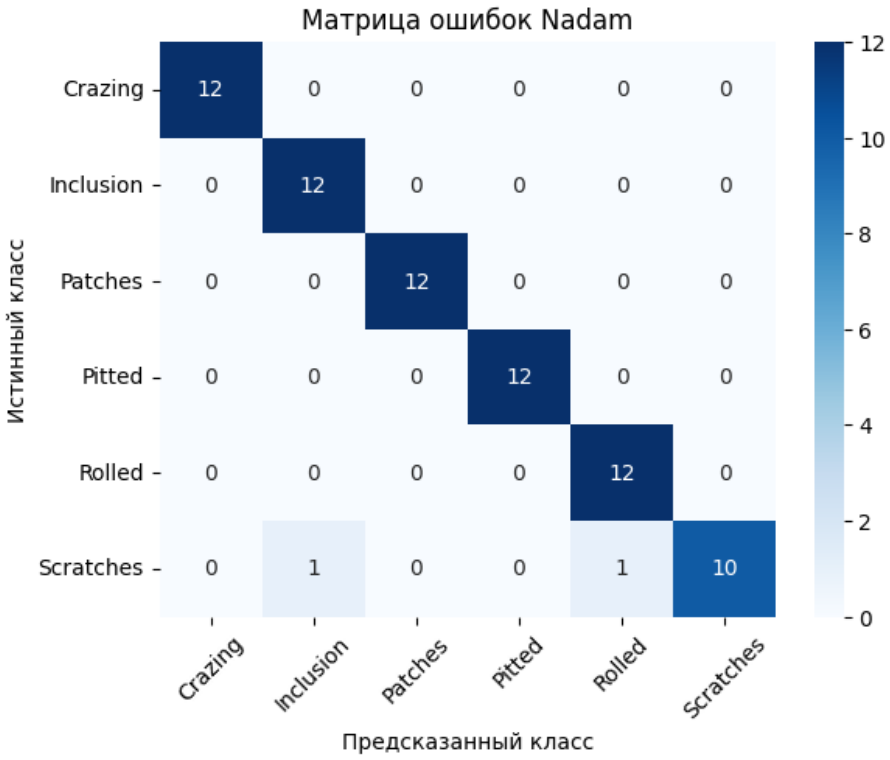

Рисунок 20 – Матрица ошибок с dropout=0,4 (Nadam)

	precision	recall	f1-score	support
Crazing	1.00	1.00	1.00	12
Inclusion	0.92	1.00	0.96	12
Patches	1.00	1.00	1.00	12
Pitted	1.00	1.00	1.00	12
Rolled	0.92	1.00	0.96	12
Scratches	1.00	0.83	0.91	12
accuracy			0.97	72
macro avg	0.97	0.97	0.97	72
weighted avg	0.97	0.97	0.97	72

Рисунок 21 – Отчет о классификации с dropout=0,4 (Nadam)

Таким образом, для Adamax оптимальным уровнем исключения нейронов является 0,3, а для Nadam – 0,4.

7 Результаты исследований

В ходе исследований были определены оптимальные параметры сети, обеспечивающие наилучшее качество классификации.

Оптимальные параметры обучения для алгоритма Adamax:

- число эпох обучения: 20;
- размер мини-батча: 32;
- уровень исключения нейронов: 0,3;
- коэффициент обучения: 0,002.

Оптимальные параметры обучения для алгоритма Nadam:

- число эпох обучения: 20;
- размер мини-батча: 16;
- уровень исключения нейронов: 0,4;
- коэффициент обучения: 0,0005.

Алгоритм Nadam показал лучшую тестовую точность, меньшее значение функции ошибки, более предсказуемую и устойчивую сходимость, чем Adamax.

Для Nadam f1-мера всех классов находится в диапазоне 0,96–1,00, а для Adamax – в диапазоне 0,89–1,00.

Метрика аккуратности при тестировании сети с оптимальными параметрами для Nadam составила 0,9861, а функция ошибки – 0,1079.

На рисунках 22 и 23 представлены отчет о классификации и матрица ошибок с оптимальными параметрами сети для алгоритма обучения Nadam.

	precision	recall	f1-score	support
Crazing	1.00	1.00	1.00	12
Inclusion	0.92	1.00	0.96	12
Patches	1.00	1.00	1.00	12
Pitted	1.00	0.92	0.96	12
Rolled	1.00	1.00	1.00	12
Scratches	1.00	1.00	1.00	12
accuracy			0.99	72
macro avg	0.99	0.99	0.99	72
weighted avg	0.99	0.99	0.99	72

Рисунок 22 – Отчет о классификации с оптимальными параметрами сети (Nadam)

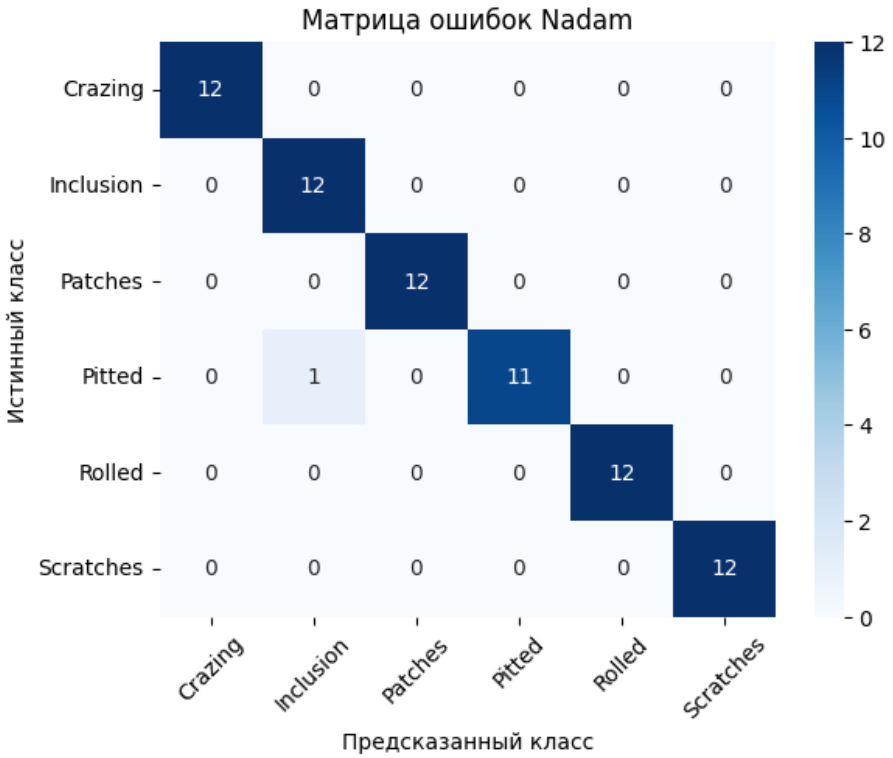

Рисунок 23 – Матрица ошибок с оптимальными параметрами сети (Nadam)

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была реализована модель свёрточной нейронной сети с использованием библиотек Keras и Tensorflow и языка Python. Эта модель может решать задачи классификации дефектов поверхности металлов.

В результате исследований определены наилучший алгоритм обучения и параметры модели, обеспечивающие наилучшую классификацию.

По результатам экспериментов установлено, что алгоритм Nadam демонстрирует лучшие показатели, чем Adamax. Наиболее удачной конфигурацией оказалась сеть Nadam с числом эпох обучения 20, размером мини-батча 16, уровнем исключения нейронов 0,4 и коэффициентом обучения 0,0005.

Данная конфигурация обеспечивает оптимальный баланс между скоростью обучения, обобщающей способностью и устойчивостью к переобучению, демонстрируя наилучшие результаты на тестовой выборке.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Metal Surface Defects Dataset [Электронный ресурс] URL: https://www.kaggle.com/datasets/fantacher/neu-metal-surface-defects-data/ (дата обращения 09.11.2025).

2 Николенко, С.И. Глубокое обучение: погружение в мир нейронных сетей [Текст] / С.И. Николенко, А.А. Кадурин, Е.О. Архангельская. – Москва: Питер, 2024. – 480 с.

ПРИЛОЖЕНИЕ А

Листинг программы

```
import os
import kagglehub
import random
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import models, layers, optimizers
from sklearn.metrics import classification_report, confusion_matrix


SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)


path += "/NEU Metal Surface Defects Data"
print("Path to dataset files:", path)


def load_dataset(path, batch_size, target_size=(200,200)):
    datagen = ImageDataGenerator(rescale=1./255)

    train_data = datagen.flow_from_directory(
        directory=os.path.join(path, 'train'),
        target_size=target_size,
        color_mode='grayscale',
        class_mode='sparse',
        batch_size=batch_size,
        shuffle=True,
        seed=SEED
    )

    valid_data = datagen.flow_from_directory(
        directory=os.path.join(path, 'valid'),
        target_size=target_size,
        color_mode='grayscale',
        class_mode='sparse',
        batch_size=batch_size,
        shuffle=False,
        seed=SEED
    )

    test_data = datagen.flow_from_directory(
        directory=os.path.join(path, 'test'),
        target_size=target_size,
        color_mode='grayscale',
```

```
        class_mode='sparse',
        batch_size=batch_size,
        shuffle=False,
        seed=SEED
    )

    return train_data, valid_data, test_data


def build_cnn_model(optimizer_name, dropout, learning_rate=0.001, input_shape=(200,200,1)):
    model = models.Sequential()
    model.add(layers.InputLayer(input_shape))

    model.add(layers.Conv2D(32, (3,3)))
    model.add(layers.PReLU())
    model.add(layers.MaxPooling2D((2,2)))

    model.add(layers.Conv2D(64, (3,3)))
    model.add(layers.PReLU())
    model.add(layers.MaxPooling2D((2,2)))

    model.add(layers.Conv2D(128, (3,3)))
    model.add(layers.PReLU())

    model.add(layers.Flatten())
    model.add(layers.Dense(128))
    model.add(layers.PReLU())

    model.add(layers.Dropout(dropout))
    model.add(layers.Dense(6, activation='softmax'))

    if optimizer_name.lower() == 'adamax':
      optimizer = optimizers.Adamax(learning_rate=learning_rate)
    elif optimizer_name.lower() == 'nadam':
      optimizer = optimizers.Nadam(learning_rate=learning_rate)

    model.compile(optimizer=optimizer, loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
    print(f"\nМодель создана: optimizer={optimizer_name}, learning_rate={learning_rate}")

    return model


def plot_training_history(history, optimizer_name, epochs):
    _, (ax1, ax2) = plt.subplots(1,2, figsize=(12,5))

    ax1.plot(history.history['accuracy'], label='Обучение')
    ax1.plot(history.history['val_accuracy'], label='Валидация')
    ax1.set_title(f'Аккуратность {optimizer_name}')
    ax1.set_xlabel('Эпоха')
    ax1.set_ylabel('Аккуратность')
    ax1.set_xticks(range(0, epochs, 5))
```

```
    ax1.legend()
    ax1.grid(True)

    ax2.plot(history.history['loss'], label='Обучение')
    ax2.plot(history.history['val_loss'], label='Валидация')
    ax2.set_title(f'Функция ошибки {optimizer_name}')
    ax2.set_xlabel('Эпоха')
    ax2.set_ylabel('Функция ошибки')
    ax2.set_xticks(range(0, epochs, 5))
    ax2.legend()
    ax2.grid(True)
    plt.tight_layout()


def plot_confusion_matrix(y_true, y_pred, class_names, optimizer_name):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure()
    sns.heatmap(cm, annot=True, cmap='Blues', xticklabels=class_names,
yticklabels=class_names)
    plt.title(f'Матрица ошибок {optimizer_name}')
    plt.xlabel('Предсказанный класс')
    plt.ylabel('Истинный класс')
    plt.xticks(rotation=45)
    plt.yticks(rotation=0)
    plt.show()


def train_and_evaluate_model(model, optimizer_name, train_data, valid_data, test_data,
epochs):
    print(f"\nОбучение модели...")
    history = model.fit(train_data, validation_data=valid_data, epochs=epochs, verbose=0)

    plot_training_history(history, optimizer_name, epochs)

    train_acc = history.history['accuracy'][-1]
    val_acc = history.history['val_accuracy'][-1]
    train_loss = history.history['loss'][-1]
    val_loss = history.history['val_loss'][-1]

    print("\nРезультаты обучения:")
    print(f"Training Accuracy:    {train_acc:.4f}, Training Loss:    {train_loss:.4f}")
    print(f"Validation Accuracy:  {val_acc:.4f}, Validation Loss:  {val_loss:.4f}")

    test_data.reset()
    test_loss, test_acc = model.evaluate(test_data, verbose=0)
    print(f"\nРезультаты тестирования:")
    print(f"Test Accuracy:       {test_acc:.4f}, Test Loss:       {test_loss:.4f}\n")

    y_pred = np.argmax(model.predict(test_data, verbose=0), axis=1)
    y_true = test_data.classes
    class_names = list(train_data.class_indices.keys())
```

```
plot_confusion_matrix(y_true, y_pred, class_names, optimizer_name)

print("\nМетрики:")
print(classification_report(y_true, y_pred, target_names=class_names))
```